Tytuł
Architektura oprogramowania

Adrian SZCZEPANSKI

ABSTRACT

Tu bedzie abstrakt

13 October 2026

1. Scheduler

Biblioteka **npos-nps** zawiera planistę zadań. Jest to planista kooperacyjny: zadanie wykonuje jedną porcję pracy w metodzie `process()` i oddaje sterowanie. Nie ma wywłaszczania.

1.1. Zadanie

Klasa `nps::Task` opisuje zadanie. Zadanie ma priorytet (liczba całkowita bez znaku) oraz trzy metody:

initialize()
: wywoływana raz przed pierwszym przebiegiem pętli,

isReady()
: informuje, czy zadanie ma coś do zrobienia,

process()
: wykonuje jedną porcję pracy.

Mniejsza wartość priorytetu oznacza wcześniejsze wykonanie. Porównanie zadań realizuje obiekt funkcyjny `nps::CompareTasks`.

1.2. Pętla planisty

Klasa `nps::Scheduler` nie przydziela pamięci. Lista zadań jest podawana przez użytkownika jako `Scheduler::TaskListInt&`, a gotowy kontener o stałym rozmiarze to `Scheduler::TaskList<N>`. Zadania dodaje się metodą `addTask()`, która zwraca `false` gdy lista jest pełna.

Metoda `start()` działa następująco:

- sortuje zadania rosnąco po priorytecie i wywołuje `initialize()` każdego z nich, następnie to samo dla następcy (planisty podłączonego przez `setSuccesor()`), zadania nadzorcy (watchdog) i zadania bezczynności (idle),
- w każdym obiegu pętli wykonuje zadanie watchdog, jeżeli jest gotowe,
- wykonuje pierwsze gotowe zadanie z listy; jeżeli żadne nie jest gotowe, pyta następcę,
- gdy nie ma gotowych zadań, wykonuje zadanie bezczynności.

Pętlę przerywa metoda `stop()`. Uwaga: wywołanie `stop()` w trakcie inicjalizacji jest ignorowane, ponieważ flaga pracy jest ustawiana dopiero po inicjalizacji.

1.3. Zadania nadzorcze

Zadania `setWatchdogTask()` i `setIdleTask()` nie trafiają na listę zadań. Planista uruchamia je poza kolejnością priorytetów, dlatego nie mają konfigurowalnego priorytetu.

2. Komunikacja międzyprocesowa

Biblioteka **npos-ipc** definiuje wiadomości, adresowanie oraz magistralę łączącą zadania.

2.1. Adresowanie

Obiekt systemu jest adresowany przez `ipc::Oid`, czyli parę (port, serwis). Port to adres zadania na magistrali, a identyfikator serwisu to indeks serwisu w zadaniu. Wartości `NULL_PORT` i `NULL_SERVICE` (255) oznaczają brak adresu. Adres `NULL_OID` oznacza rozgłaszanie.

Oid wskazuje wyłacznie na obiekty, które mogą zawieść z powodu sprzętu lub plików, na przykład otwarte urządzenie albo otwarty plik. Zadania i serwisy nie są obiektami.

2.2. Wiadomość

Struktura `ipc::Message` ma stały rozmiar. Zawiera:

sender
: port nadawcy,

recipient
: adres odbiorcy (Oid),

type
: typ wiadomości,

referenceCount
: licznik odwołań, gdy wiadomość jest przechowywana w puli,

input
: dane wejściowe: unia pól typowanych (otwarcie, atrybuty, odczyt lub zapis, połączenie) oraz wskaźnik i rozmiar bufora,

output
: analogiczna część wyjściowa ze statusem odpowiedzi.

Typy wiadomości są podzielone na zakresy: błędy i logi systemowe, urządzenia, sterowniki, system plików, cli oraz użytkownika. Konkretne wiadomości dziedziczą po `ipc::Message` i dodają wyłącznie akcesory, bez nowych pól. Bufory nie są kopiowane, więc muszą istnieć do czasu obsłużenia odpowiedzi.

2.3. Serwis

Serwis (`ipc::Service`) implementuje metody `handle()`, `accepts()` oraz opcjonalnie `initialize()`. Komunikuje się przez interfejs `Service::Api`: `send()`, `respond()` i `getPortId()`. Obiekt `Api` jest podawany w konstruktorze, a implementuje go zadanie serwisów.

`ipc::FsmService` to serwis będący automatem skończonym. Stany dziedziczą po `FsmService::State` (metody `onEnter()`, `onExit()`, `accepts()`, `handle()` i `handleUnknown()`). Obsługa zwraca identyfikator następnego stanu albo `NO_STATE_CHANGE`. Serwis akceptuje typy wiadomości wyłącznie bieżącego stanu, a po `start()` wchodzi w stan o indeksie 0.

2.4. Zadanie serwisów

Klasa `ipc::ServiceTask` jest jednocześnie zadaniem, portem na magistrali i `Service::Api`. Listę serwisów i kolejkę wiadomości dostaje od użytkownika jako interfejsy `ServiceListInt&` i `MessageQueueInt&`.

- `subscribe()` nadaje serwisowi identyfikator równy indeksowi w liście,
- wiadomość do tego samego portu jest obsługiwana od razu, bez kolejkowania,
- wiadomość do innego portu trafia na magistralę,
- odbiorca z konkretnym identyfikatorem serwisu dostaje wiadomość bezpośrednio, a przy `NULL_SERVICE` wiadomość trafia do wszystkich serwisów, które akceptują jej typ.

Odpowiedzi są adresowane po polu `sender` z żądania. Serwis, który tylko czeka na odpowiedzi, powinien rozróżniać własne żądania od cudzych po porcie nadawcy.

2.5. Magistrala

Interfejs `ipc::Bus` ma metody `send()`, `respond()` i `release()`. Implementacja `ipc::SystemBus` przyjmuje listę portów oraz pulę wiadomości od użytkownika. Numery portów są nadawane od wartości początkowej, więc dwie magistrale mogą mieć rozłączne zakresy i być połączone jako następcy.

- Wiadomość spoza puli jest kopiowana do puli przed doręczeniem.
- Każde doręczenie zwiększa licznik odwołań, a `release()` zmniejsza go i zwraca wiadomość do puli, gdy licznik spadnie do zera.
- Rozgłaszanie omija port nadawcy i porty, które nie akceptują typu wiadomości.
- Odpowiedź wraca do portu zapisanego w polu `sender`.
- Nieznany port jest przekazywany następcy, o ile istnieje.

Usługa `OS_SYSLOG_MESSAGE_TYPE` służy do przesyłania logów, które wyświetla terminal.

3. Urządzenia

Biblioteka **npos-dev** udostępnia urządzenia sprzętowe jako serwisy. Każde urządzenie ma interfejs warstwy sprzętowej (HAL) w `hal/`, wiadomości w `msg/`, serwis w `service/` oraz, dla zasobów systemowych, zadania w `task/`.

3.1. Otwieranie urządzeń

Urządzenie jest identyfikowane nazwą. Klient wysyła rozgłoszeniowe `dev::msg::Open` (funkcja `dev::api::open()`). Odpowiada serwis o pasującej nazwie, przekazując w `getOpened()` swój Oid. Dalsze żądania są kierowane już bezpośrednio do tego Oid, dzięki czemu nie obciążają magistrali rozgłoszeniami. Serwis o innej nazwie milczy.

3.2. Serwisy

Serwis	HAL	Wiadomość
LedHandler	**hal::Led**	**msg::Led**
RtcClock	**hal::Rtc**	**msg::Clock**
BackupStore	**hal::BackupRegisters**	**msg::BackupRegisters**
UartPolling	**hal::UartPolling**	**msg::Transciever**
UartInterrupt	**hal::UartInterrupt**	**msg::Transciever**
HashProcessor	**hal::Hash**	**msg::Hash**

Serwisy oparte na przerwaniach dziedziczą po `dev::InterruptService`. Przerwanie trafia do kolejki przerwań podanej przez użytkownika, a zadanie `dev::InterruptServiceTask` obsługuje je przed zwykłymi wiadomościami. Żądanie, które trwa dłużej niż jedno wywołanie, jest przechowywane przez serwis ze zwiększonym licznikiem odwołań i oddawane po przerwaniu.

Transciever (`dev::msg::Transciever`) służy do odczytu i zapisu w urządzeniach szeregowych. Odczyt prosi o *do* N bajtów. Urządzenie może zwrócić mniej i wtedy podaje faktyczną liczbę przez `setOutputLength()`.

3.3. Zadania systemowe

Timer
: przyjmuje `dev::msg::Timer` (jednorazowy lub powtarzalny) i odpowiada po upływie interwału, korzystając z `hal::SystemTick`.

Idle usypia procesor przez `hal::Power` gdy nie ma pracy.

Watchdog
: odświeża układ nadzorujący w każdym obiegu planisty.

Idle i Watchdog mają stały priorytet 0 i są uruchamiane przez planistę poza listą zadań.

4. System plików

Biblioteka **npos-fs** definiuje interfejs systemu plików, a zadanie `fs::task::Littlefs<N>` implementuje go na bibliotece littlefs. Parametr N to liczba jednocześnie otwartych plików oraz, osobno, katalogów.

4.1. Obsługa sprzętu

Pamięć opisuje `fs::hal::Flash` (blokowy dostęp do pamięci flash) i opcjonalnie `fs::hal::ICache` (pamięć podręczna instrukcji jest wyłączana na czas programowania). Jeżeli montowanie się nie powiedzie, zadanie formatuje pamięć i montuje ją ponownie.

4.2. Wiadomości

Wiadomość	Znaczenie
file::Open	**otwarcie pliku z flagami (READ_ONLY, WRITE_ONLY, CREATE, TRUNCATE, APPEND, EXCLUSIVE)**
file::Read, file::Write	**odczyt i zapis do bufora użytkownika**
file::Close	**zamknięcie pliku**
dir::Open, dir::Close	**otwarcie i zamknięcie katalogu**
dir::Read	**odczyt kolejnego wpisu (typ, rozmiar, nazwa do 64 znaków)**
dir::Make	**utworzenie katalogu**

Otwarcie zwraca Oid pliku lub katalogu. Pole identyfikatora serwisu w tym Oid to indeks slotu w zadaniu. Odczyt katalogu kończy status `END_OF_DIRECTORY`; wpisy `.` i `..` nie są zwracane. Brak wolnego slotu skutkuje odpowiedzią `NO_MEMORY`.

Ścieżki w żądaniach otwarcia i utworzenia katalogu muszą być zakończone zerem, ponieważ trafiają bezpośrednio do littlefs. Funkcje `fs::api::file::*` i `fs::api::dir::*` wysyłają żądania w imieniu serwisu.

5. Terminal

Biblioteka **npos-cli** zawiera terminal tekstowy oraz programy uruchamiane z jego linii poleceń.

5.1. Terminal

Zadanie `cli::task::Terminal` (wzorowane na zadaniu timera: zadanie, port i kolejka wiadomości) otwiera urządzenie wskazane nazwą i wysyła do niego `Transciever` w trybie odczytu na do 64 bajtów. Każdy odebrany bajt jest naciśnięciem klawisza. Wyjście jest kolejkowane i wysyłane porcjami, a gdy poprzedni zapis jeszcze trwa, dane czekają w buforze.

- Edycja linii: backspace, Delete, Home, End, strzałki lewo i prawo, Ctrl-A, Ctrl-E, Ctrl-U (kasuje linię), Ctrl-C (porzuca linię). Rozpoznawane są sekwencje escape CSI i SS3.
- Terminal prowadzi bufor linii i położenie kursora oraz robi lokalne echo.
- Historia pamięta pięć ostatnich poleceń, a strzałki w górę i w dół ją przełączają, przywracając linię rozpoczętą przed wejściem w historię.
- Log systemowy odebrany jako `OS_SYSLOG_MESSAGE_TYPE` jest wypisywany nad edytowaną linią: terminal kasuje linię, wypisuje log i rysuje linię ponownie wraz z kursorem.
- Terminal śledzi bieżący katalog (najwyżej 64 znaki) i pokazuje go w znaku zachęty.

Terminal jest własnym zadaniem i własnym portem na magistrali, a jednocześnie `Service::Api` swoich programów, więc dostaje wszystkie odpowiedzi do swoich żądań niezależnie od identyfikatorów serwisów w innych zadaniach. Odpowiedzi na cudze żądania są pomijane po porcie nadawcy.

Tekst logu leży w buforze nadawcy, który istnieje tylko w czasie wysyłania. Dlatego przy odbiorze terminal kopiuje go do slotu puli (`PayloadPoolInt`), podmienia `input.data` i dopiero wtedy kolejkuje wiadomość. Slot jest zwalniany po obsłużeniu logu. Gdy brakuje slotu, tekst jest za długi albo kolejka jest pełna, log jest odrzucany.

5.2. Programy

Program (`cli::Program`) jest serwisem. Terminal nie subskrybuje go do zadania, tylko przekazuje mu odpowiedzi działającego programu. Program implementuje ,`start(host,`args) odpowiedzi w `handle()` i opcjonalnie `resume()`, wywoływane gdy wyjście terminala zostało opróżnione. Terminal udostępnia programom interfejs `Program::Host`: `print()`, `exit()` oraz odczyt i zmianę katalogu bieżącego. Wydruk zwraca liczbę przyjętych znaków, a program kontynuuje po `resume()`, dzięki czemu wypisanie dużego pliku nie wymaga dużego bufora.

Ścieżki względne są rozwiązywane względem katalogu bieżącego terminala funkcją `text::resolve()`, która obsługuje `.` i `...`

Program	**Działanie**
cat <plik>	wypisuje plik
echo <tekst> [> | >> <plik>]	wypisuje tekst albo zapisuje go do pliku
touch <plik>	tworzy pusty plik
ls [katalog]	wypisuje wpisy katalogu
cd [katalog]	zmienia katalog, bez argumentu przechodzi do /
pwd	wypisuje katalog bieżący
mkdir <katalog>	tworzy katalog

6. System

Biblioteka **npos-os** składa cały system z interfejsów HAL według schematu z przykładów.

6.1. Pamięć

System nie przydziela własnych kontenerów. Użytkownik podaje strukturę `os::Memory` ze referencjami do interfejsów: lista zadań, lista portów, pula wiadomości, pula serwisów urządzeń, listy serwisów i kolejki wiadomości zadań oraz kolejka timera kolejka systemu plików, kolejka terminala oraz pula slotów na teksty logów terminala. Gotowy zestaw o ustalonych rozmiarach daje szablon `os::Storage<Zadania, Serwisy, Kolejka, Wiadomości, SerwisyUrządzeń, Programy>`, którego metoda `memory()` zwraca `Memory`. Pamięć musi przeżyć system.

Serwisy urządzeń są tworzone w puli o slotach wielkości największego z nich (`os::ServiceSlot::SIZE`). Serwisy, które nie zmieszczą się w puli, są pomijane.

6.2. Sprzęt

Struktura `os::Hal` jest tworzona z obowiązkowym `SystemTick&` i opisuje pozostałe urządzenia:

flash,icache
: dodają zadanie systemu plików,

power
: dodaje zadanie idle,

watchdog
: dodaje zadanie watchdog,

leds (`LedBank`),

rtcs,backups,pollingUarts
: (`Device<T>` z nazwą),

uarts,hashes
: (`InterruptDevice<T>` z nazwą i kolejką przerwań).

Urządzeń każdego rodzaju może być wiele, każde pod własną nazwą.

6.3. Konfiguracja i uruchomienie

Struktura `os::Config` zawiera flagę `cli`, nazwę urządzenia terminala oraz priorytety zadań: użytkownik 1, terminal 2, system plików 4, urządzenia 5, urządzenia z przerwaniami 6, timer 9. Wszystkie mają wartości domyślne.

Serwisy użytkownika powstają przed systemem, więc potrzebują obiektu `Api` już w konstruktorze. Rozwiązuje to `os::UserApi`, który system wiąże z zadaniem użytkownika. Tworzenie systemu wygląda tak:

```
os::Storage<> storage;
os::UserApi api;
MyService service(api);
ipc::Service* services[] = { &service };

os::Hal hal(tick);
hal.rtcs = rtcs;

os::System system(hal, storage.memory(), services, api);
system.start();
```

System tworzy zadania: użytkownika, urządzeń bez przerwań, urządzeń z przerwaniami, systemu plików (gdy podano pamięć flash), timera oraz idle i watchdog (gdy podano odpowiednie HAL). Zadania bez serwisów nie są dodawane do magistrali. Klasa `os::System` to alias `os::BasicSystem<8>`; parametr szablonu to liczba otwartych plików.

7. Emulator

Biblioteka **npos-emulator** zawiera implementacje HAL do uruchamiania systemu na komputerze: Flash, ICache, Led, Rtc, SystemTick, Hash, UartInterrupt, UartPolling, BackupRegisters, Power, Watchdog oraz ConsoleUart (stdin i stdout).

Mocki, które coś wypisują, mają argument konstruktora `verbose` (domyślnie wyłączony). Wydruki są włączane tylko w programach przykładowych, a testy jednostkowe są ciche.

ConsoleUart zwraca dane tak szybko, jak są dostępne, a resztę bufora wypełnia zerami, co terminal ignoruje.

8. Testy i przykłady

8.1. Testy jednostkowe

Każda biblioteka ma katalog `tests` z testami w CppUTest. Testy są uruchamiane przez Makefile z głównego katalogu przy budowaniu biblioteki, a niepowodzenie testu przerywa budowanie.

- Plik testowy modułu ma nazwę `Moduł.test.cpp`, a pliki są pogrupowane w katalogach jak w bibliotece.
- Pliki obiektowe są nazywane bez katalogu, więc gdy dwa moduły mają tę samą nazwę, do nazwy testu dodaje się przedrostek (na przykład `DirOpen.test.cpp`).
- Atrapy znajdują się w `tests/mock`.
- Nagłówki projektu należy dołączać przed `CppUTest/TestHarness.h`, ponieważ ten nagłówek podmienia operator `new`.

8.2. Przykłady

Katalog `examples` zawiera programy korzystające z `os::System` i emulatora: blinking-led, get-time, compute-hash, uart-read-write, filesystem, dual-system oraz cli (terminal na stdin i stdout, na przykład `printf 'ls\n' | ./bin/Pc32/debug/cli.elf`). Wskazówka: polecenia podawane przez potok należy rozdzielać pauzami, ponieważ terminal ignoruje bajty wpisane w trakcie działania programu.

Table of Contents

1. Scheduler . . . 3
   1.1. Zadanie . . . 3
   1.2. Pętla planisty . . . 3
   1.3. Zadania nadzorcze . . . 3
2. Komunikacja międzyprocesowa . . . 3
   2.1. Adresowanie . . . 3
   2.2. Wiadomość . . . 3
   2.3. Serwis . . . 4
   2.4. Zadanie serwisów . . . 4
   2.5. Magistrala . . . 4
3. Urządzenia . . . 4
   3.1. Otwieranie urządzeń . . . 5
   3.2. Serwisy . . . 5
   3.3. Zadania systemowe . . . 5
4. System plików . . . 5
   4.1. Obsługa sprzętu . . . 5
   4.2. Wiadomości . . . 5
5. Terminal . . . 6
   5.1. Terminal . . . 6
   5.2. Programy . . . 6
6. System . . . 6
   6.1. Pamięć . . . 7
   6.2. Sprzęt . . . 7
   6.3. Konfiguracja i uruchomienie . . . 7
7. Emulator . . . 7
8. Testy i przykłady . . . 8
   8.1. Testy jednostkowe . . . 8
   8.2. Przykłady . . . 8